Modeling Human Activity States Using Hidden Markov Models

Formative 2: Human Activity Recognition for inactivity and sudden movement monitoring

Author: Eddy Irasetsa | Repository:
https://github.com/Eddydev-ALU/hmm-human-activity-recognition-formative-2

1. Background and Motivation

Many older adults live alone. Two events are dangerous if nobody notices them quickly: a long period without movement (for example after a fall or sudden illness) and a sudden, violent movement such as a fall or a seizure. My use case is a phone or wearable that tracks the wearer's activity from its accelerometer and gyroscope and alerts a caregiver when either event happens. Activity recognition is needed because the sensor signal is noisy and the activity itself is never measured directly; it has to be inferred from signal patterns and from how activities follow each other. A Hidden Markov Model fits this well: the activity is a hidden state that tends to persist, and each short window of sensor data is a noisy observation of it. I use four states: standing, walking, still (the inactivity alarm) and jumping, which is a safe way to record a sudden high-energy movement in place of a real fall or seizure.

2. Data Collection and Pre-processing

Table 1. Devices and sampling rates.

Person	Phone	App / OS	Configured rate	Detected rate (from timestamps)
Eddy Irasetsa	iPhone XS	Sensor Logger 1.68 (iOS 18.7.5)	50 Hz (sampleRateMs = 20)	49.76–49.77 Hz → resampled to 50 Hz
A friend (shared test data used for testing purpose)	Samsung SM-A235F	Sensor Logger 1.68 (Android)	100 Hz (sampleRateMs = 10)	102.07 Hz → low-pass + resampled to 50 Hz

I recorded accelerometer and gyroscope data (x, y, z with timestamps) using Sensor Logger and exported each recording as CSV. For standing, walking and jumping the phone was held at waist level; for still it lay on a flat table. There are 52 training files, 13 per activity. After cutting 2 s from each end (the start and stop taps), each file has 8.7–12.5 s of clean data, and every activity has well over the required 90 s (Table 2). Accelerometer.csv holds acceleration with gravity removed (m/s²) and the gyroscope gives angular velocity (rad/s). A friend (not a group member; this assignment was done individually) shared one recording per activity from their own phone so I had extra unseen test data. These files were used only for testing.

Table 2. Training data per activity.

Activity	Files	Raw seconds	Clean seconds	2 s windows
Standing	13	179.1	127.3	108
Walking	13	177.7	125.9	107
Jumping	13	176.9	125.1	106
Still	13	175.5	123.7	104
Total	52	709.2	502.0	425

Cleaning. The label comes from the folder name, and unexpected labels are rejected. Columns are selected by name, because the export order is z, y, x. The two sensors are merged on the nearest timestamp. Rows with missing values are dropped (there were none). Raw files are never changed; cleaned files go to data/processed/.

Different sampling rates. I measured each file's rate from the median time step instead of trusting the app setting: 49.76–49.77 Hz on the iPhone and 102.07 Hz on the Samsung. Every file is resampled onto the same 50 Hz grid by linear interpolation. For the 100 Hz phone, a 20 Hz low-pass filter (4th-order Butterworth, zero-phase) is applied first so that fast vibrations do not alias. Without this step a 100-sample window would last 1 s on one phone and 2 s on the other, and the frequency features would not be comparable.

Window size. The window length is computed from the sampling rate: 2 s × 50 Hz = 100 samples, with 50 % overlap, so the model gets one observation per second. Walking (about 2 steps/s) and jumping (about 1.5–2 jumps/s) then show at least two full cycles per window. The FFT resolution of 50/100 = 0.5 Hz is enough to separate the walking peak (1–2.5 Hz) from the jumping peak (3.5–4 Hz). A 1 s window would hold less than two cycles, and a 4 s window would blur the moments where the activity changes.

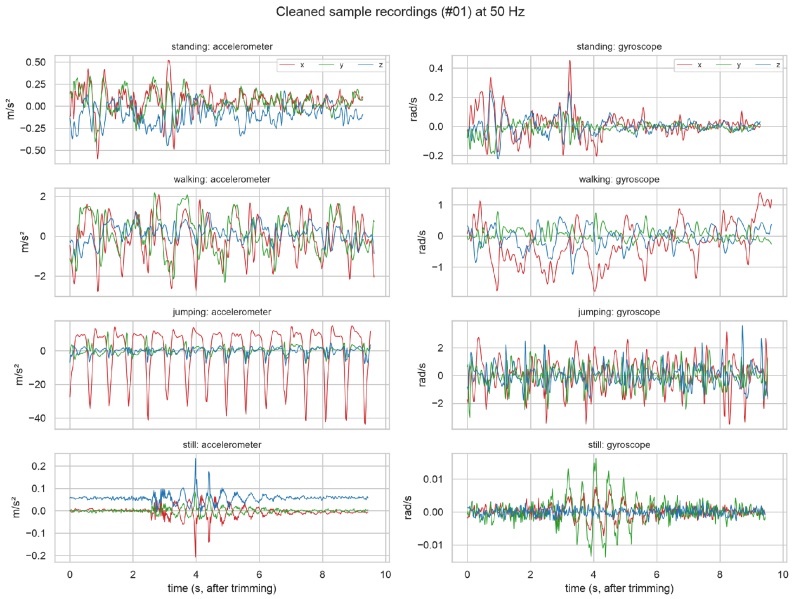


Figure 1. Cleaned accelerometer (left) and gyroscope (right) signals, one recording per activity, at 50 Hz. The y-axis scales differ a lot, from about ±0.1 m/s² (still) to about 40 m/s² (jumping).

Videos (one per activity):

- **Standing:** https://drive.google.com/file/d/1f4_p_-ujCiJZu97DpZ5USixrpc7iwrEp/view?usp=sharing
- **Walking:** https://drive.google.com/file/d/1jgML2SbKMkt49ShfATg1yWFIp1IBQ9B9/view?usp=sharing
- **Jumping:** https://drive.google.com/file/d/1U_uXRKpqcJjFT1HDZSg_uy9OPgarakPx/view?usp=sharing
- **Still:** https://drive.google.com/file/d/1k5gLUf_np0WNJqVYsKyLhyXlgU_nX-zk/view?usp=sharing

3. HMM Setup and Implementation Details

Model. Hidden states Z are the four activities. Each observation X is the 17-value feature vector of one window. A (4×4) holds the probability of moving from one activity to the next, π the probability of the first activity, and B is one Gaussian per state with a diagonal covariance. I chose diagonal covariance because a full 17×17 covariance needs 153 values per state from only about 100 windows. A variance floor of 0.01 stops the very steady still features from producing extreme likelihoods.

Table 3. Features (12 time-domain, 5 frequency-domain).

Feature	Domain	Why it helps separate the activities
acc_rms, gyro_rms (log)	Time	Overall movement and rotation strength: still ≪ standing < walking ≪ jumping.
acc_std_x/y/z, gyro_std_x/y/z (log)	Time	Movement per axis: jumping is mostly vertical, walking spreads over all axes and adds hip rotation.
acc_sma (log)	Time	Signal magnitude area, a standard activity-energy measure that does not depend on how the phone is turned.
acc_corr_xy/xz/yz	Time	Axes move together during jumps; sensor noise on a still phone is uncorrelated.
acc_dom_freq, gyro_dom_freq	Freq. (FFT)	Main frequency: walking 1–2.5 Hz, jumping 3.5–4 Hz (two impacts per jump), random for still.
acc_spec_energy (log)	Freq. (FFT)	Total power of the movement.
acc/gyro_spec_entropy	Freq. (FFT)	Regular motion gives a peaked spectrum (low entropy), noise gives a flat one (high entropy).

Normalisation. Energy features range over about three orders of magnitude, so I take their log10 first. Then every feature is Z-scored, $z = (x - \mu)/\sigma$, with μ and σ from the training windows only (no test data leaks in). Z-scoring puts features with different units (m/s², rad/s, Hz) on the same scale so none dominates the Gaussian likelihood, and unlike min-max scaling it is not squashed by a few large jump spikes.

Algorithms and metrics. The HMM is written from scratch in numpy, in log space to avoid underflow. The forward algorithm gives $\log P(X \mid \lambda)$, which I use to check convergence. Forward-backward gives the state probabilities that Baum-Welch needs to learn π, A and B. Viterbi returns the single most likely activity for each window. Sensitivity, specificity, accuracy and the confusion matrix all need one hard label per window, so Viterbi is the right decoder for this evaluation. Unit tests check Viterbi and the forward algorithm against brute force on small random models.

Training. Each training file holds only one activity, so it has no transitions. I joined recordings into 13 sequences, each with one recording of every activity in random order. Means, variances, A and π were first estimated from the labels (with +1 smoothing so no probability is zero). Baum-Welch then refined everything without labels, stopping when the log-likelihood gain fell below 10^{-4}. It converged in 6 iterations, from 675.1 to 826.4 (Figure 2). States were matched to activity names with the Hungarian algorithm; every state kept its original activity.

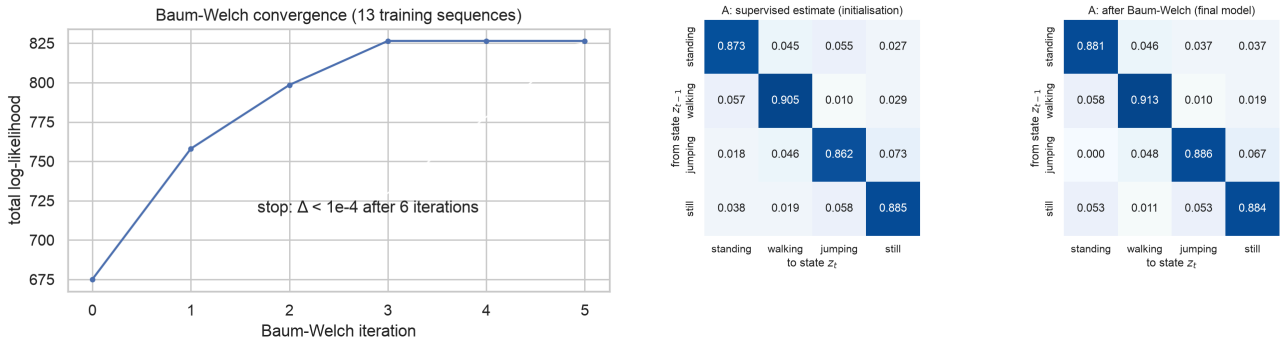


Figure 2. Left: Baum-Welch log-likelihood per iteration. Right: transition matrix A before and after Baum-Welch.

4. Results and Interpretation

Unseen data. No test file was used for training or for any design choice. (a) Same phone, new session: about 20 minutes after training I recorded three longer files with all four activities back to back (standing→walking→jumping→still twice, and still→jumping→walking→standing once), so they contain real transitions. I labelled the boundaries by hand from the movement trace and left out the 8 windows where the phone was being put down or picked up. (b) Different person, phone, sampling rate and day: the friend's four single-activity files from the Samsung (100 Hz). In total there are 108 test windows (88 + 20).

Table 4. Results on all unseen test data. Per-class values are one-vs-rest; the last row is the multi-class accuracy.

State (Activity)	Number of Samples (windows)	Sensitivity	Specificity	Overall Accuracy
Standing	23	0.870	0.929	0.917
Walking	34	1.000	0.932	0.954
Jumping	32	0.938	1.000	0.982
Still	19	0.684	1.000	0.944
All (overall)	108	0.873 (macro)	0.966 (macro)	0.898

Overall accuracy on unseen data is 89.8%, compared with 98.4% on the training data. It is 87.5% on the same-phone sequences and 100.0% on the friend's phone (20 of 20 windows). Walking was never missed (sensitivity 1.00) and jumping was almost always right (sensitivity 0.94, specificity 1.00). Still was the weakest class: 13 of 19 windows were found (sensitivity 0.68), and all 6 misses were labelled standing. Its specificity is 1.00, so when the model says still, it is right. The emission means (Figure 3, right) show why most classes are easy: on the energy features the four states sit far apart, in the order still < standing < walking < jumping. The decoded sequence (Figure 4) shows where the errors are: 10 of the 11 errors are within 2 s of an activity change.

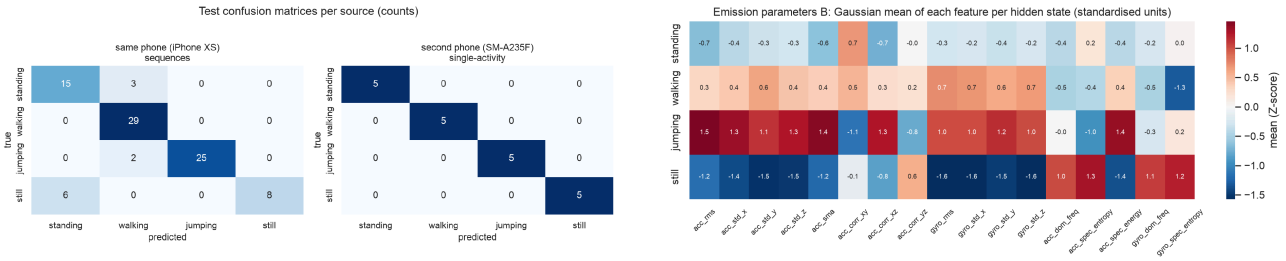


Figure 3. Left: test confusion matrices for (a) the same-phone sequences and (b) the friend's phone. Right: emission parameters B (mean of each feature per state, in Z-units).



Figure 4. True (green) vs Viterbi-decoded (red) activity for one unseen recording, with acceleration magnitude on top. Crosses mark errors. Plots for all 7 test files are in the notebook.

5. Discussion and Conclusion

Easiest and hardest activities. Jumping and walking were easiest: their energy is 10–100 times higher than the static classes and their spectra have clear peaks. The only mistakes were 2 jumping windows labelled walking where a jump bout ended. Still vs standing was hardest. iOS removes gravity from Accelerometer.csv, so the model cannot see that the phone is flat on a table rather than upright at the waist, and has to rely on the small difference between body sway and pure sensor noise. The missed still windows sit right next to the phone being put down or picked up (one just after the start tap), when it is still moving slightly. Standing was labelled walking 3 times, each time in a window that overlaps the start or end of walking. For the use case this is acceptable: a late still label only delays an inactivity alarm by a second or two, and the sudden-movement class was caught from its first window.

Do the transition probabilities look realistic? Partly. The high self-transitions (0.88–0.91) are realistic because activities last a while; with a 1 s step they mean bouts of about 8–11 s. But that comes from my short recordings; in daily life people stay still or standing for minutes. The off-diagonal values come from the random order I used to join the recordings, so they say little about real behaviour. A realistic A would need long recordings of normal daily routines.

Effect of sensor noise and sampling rate. Noise matters most for the static classes. Still accelerometer noise is about 0.005 m/s², so still features are very tight; without the variance floor any slightly noisier still window would be rejected. The Samsung was quieter than the iPhone when still (mean log10 acc_rms -1.80 vs -1.24) and sampled at 102 Hz instead of 50 Hz. After resampling, these differences were much smaller than the gaps between classes, so the model worked on it without retraining. 50 Hz is far above the 1–4 Hz movement frequencies, so the lower rate lost nothing. The 2 s window does limit timing to about 1–2 s, which is exactly where the boundary errors happen.

Generalisation. The model generalises well. It was perfect on a different person and phone even though their walking rhythm differed (1.5 vs 2.0 Hz), although that set is small and has no transitions. The drop from 98.4% to 87.5% on the same-phone sequences comes almost entirely from activity changes, which never appear in the training files. For comparison, the HMM before Baum-Welch scored 93.5% and a classifier without transitions 89.8%. I report these as they came out and did not tune anything on the test data.

Improvements. More varied data (more people, phone positions and places, and long recordings with natural transitions); the gravity or raw accelerometer signal to tell still from standing; extra features such as jerk and zero-crossing rate; an explicit transition state or duration model; and shorter windows for sharper boundaries.

Conclusion. I collected 52 labelled recordings, brought them to a common 50 Hz rate, extracted 12 time-domain and 5 frequency-domain features, and built a Gaussian HMM from scratch with Viterbi and Baum-Welch. On unseen data it reached 89.8% accuracy, with near-perfect walking and jumping detection. Its main weakness is telling still from standing right after an activity change. The next step for an elderly-monitoring system is adding phone orientation and long real-life recordings.